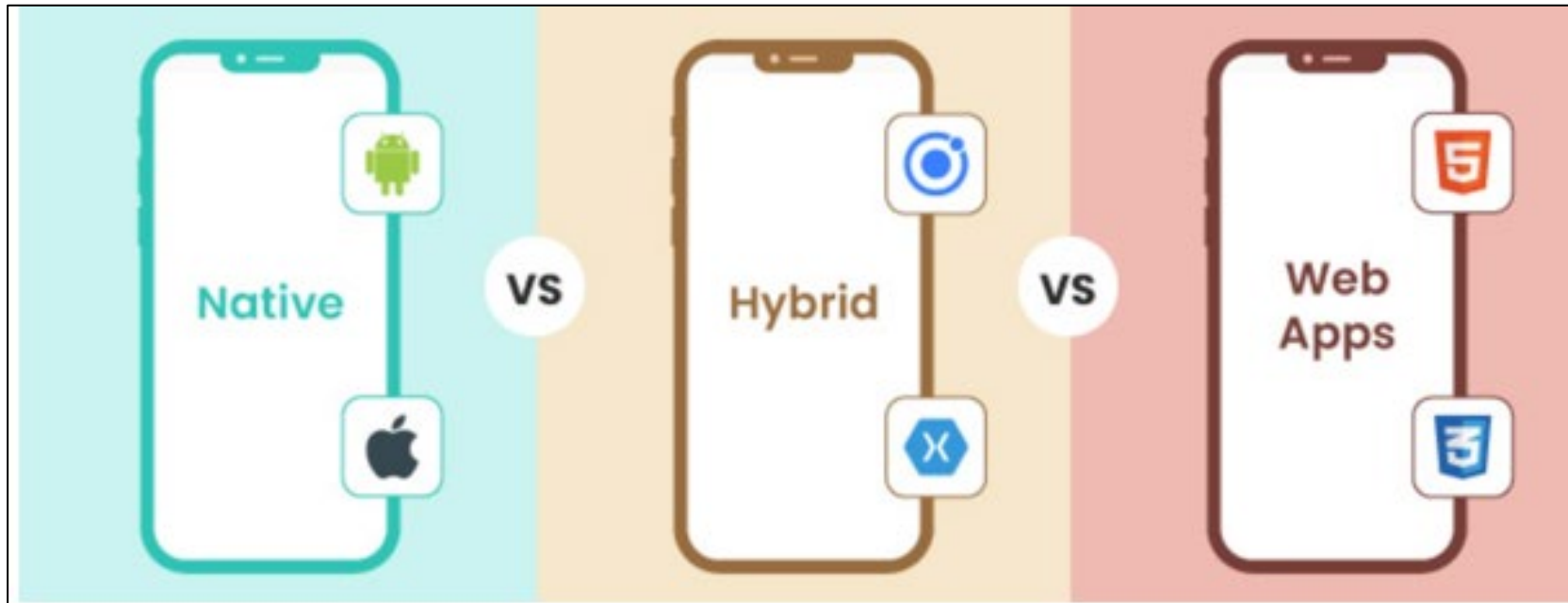

PROGRAMMING FOR MOBILE APPLICATIONS

ELC - 425A



Prepared By: Dr. Walaa Omar Badr

Types of Mobile Applications



Native applications

1. Native applications are designed specifically for a mobile operating system, typically iOS or Android.
2. All apps targeted toward a particular mobile platform **So, an app designed for Android mobile will not work on the iOS platform.**
3. The main purpose for creating such apps is to ensure best performance for a specific mobile operating system.

Examples:

WhatsApp (Android & iOS native versions)

☐ **Technology Used for Developing Native Apps**

- **C++**
- **Java**
- **Python**
- **React Native**

When to choose native app development?

- **Choose native app development for high performance and app stability.**
- **If you want to code specifically to only one platform.**
- **Need to add device-specific functionalities like GPS, camera, and so on, native app development is the choice.**
- **If you have time, money, and resources for quick app development.**

Disadvantages:

- Must build separate apps for Android and iOS
- Higher development cost

Web based apps

- ❑ **Web apps typically run through a browser. (Runs in browser like Google Chrome or Safari)**
- ❑ **Internet access is typically required for proper behavior or being able to use all features**
- ❑ **Web applications include all browser-supported applications. Additionally, you can only access web applications with a stable internet connection.**
- ❑ **Example:**
 - Facebook mobile website
 - Netflix
- ❑ **Technology Used for Developing Web-based Apps**
 - Ruby
 - Python

When to choose web based app development?

- **Web-apps are the easiest to test and deploy.**
- **These types of mobile apps are also the fastest to build.**
- **If you have a fixed budget with a few mobile functionalities.**
- **Web app development is easy if you do not want to manage multiple platform codebases.**

- **Lower performance than native apps.**
- **Requires internet**

Hybrid Apps

- ❑ These are made to support web and native technologies across multiple platforms.
- ❑ These apps are easier and faster to develop.
- ❑ It involves use of single codebase which works in multiple mobile operating systems
- ❑ For example: **Instagram & Amazon** & Uber
- ❑ Technology Used for Developing Hybrid Apps(Using frameworks like:)
 - Flutter
 - JavaScript
 - Xamarin
 - Ionic
 - React Native

When to choose hybrid app development?

- If you have limited development time and monetary resources.
- Hybrid app development provides good performance with fewer development roadblocks.
- Provides ease of maintaining a single codebase.
- Unless you build an application heavily dependent on performance, like a gaming application, you must go with the hybrid model.

If the app needs:

- Very fast processing
- High graphics quality
- Real-time response
- Strong CPU/GPU usage

Examples:

- 3D games like PUBG: Battlegrounds
- High-end games like Call of Duty: Mobile
- AR/VR apps

These apps usually need **native development** for maximum speed and performance.

If your app is something like:

- E-commerce app
- Chat app
- News app
- Booking app

Then using hybrid frameworks like:

- Flutter
- React Native

is usually better because:

- It's cheaper
- Faster to build
- One codebase works for Android & iOS

In simple words

- High-performance app (games, AR) → **Native**
- Normal business app → **Hybrid is enough**

Mobile app development is the act or process by which a mobile app is developed for one or more mobile devices, which can include personal digital assistants (PDA), enterprise digital assistants (EDA), or mobile phones. Such software applications are specifically designed to run on mobile devices, taking numerous hardware constraints into consideration. Common constraints include CPU architecture and speeds, available memory (RAM), limited data storage capacities, and considerable variation in displays (technology, size, dimensions, resolution) and input methods (buttons, keyboards, touch screens with or without styluses).

Mobile app development is the process of designing, building, testing, and deploying applications for mobile devices.

What does mobile app development include?

[1] Designing

Creating the app layout (buttons, screens, colors).

[2] Coding (Programming)

Writing the code using:

- Java / Kotlin (Android)
- Swift (iOS)
- Or frameworks like Flutter

[3] Testing

Checking if the app works correctly.

[4] Publishing

Uploading the app to the app store.

Dart Tutorial

❖ Dart is an open-source general-purpose programming language developed by Google.

This means:

Dart can be used for many types of applications:

- Mobile apps

- Web apps

- Desktop apps

- Backend servers

❖ It supports application development on both the client and server side.

❖ It is widely used for the development of Android apps, iOS apps, IoT(Internet of Things), and web applications using the Flutter Framework

❖ Dart is quite similar to that of Java Programming.

❖ Dart is extensively used to create single-page websites and web applications. The Best example of a dart application is Gmail.

❖ Dart language was released in 2011 but came into popularity after 2015 with Dart 2.0.

Why Use Dart

- ❑ **Fast & Smooth:** Dart compiles to native code for speedy performance, ideal for mobile apps.
- ❑ **Easy to Learn:** Similar to familiar languages like Java or Java script, making it simple for new developers.
- ❑ **Flutter Power:** Dart is the heart of Flutter, a popular framework for building beautiful and functional mobile apps.
- ❑ **One Code, Many Places:** Develop for mobile, web, and even desktop with a single codebase.

Features of Dart Programming Language

- 1) **Dart is a versatile language known for its simplicity and efficiency in developing applications for various platforms.**
- 2) **Easy to Understand: Dart because of its similarities with C# and Java user often find the Dart Syntax Familiar. Also, as it provides the code Reuse it makes the Programs clean and easy to understand.**
- 3) **Object Oriented Programming: Nowadays OOPS is considered as one of the most important features for many Programming Language including Java and C++.**
- 4) **Open Source: As Dart is Open Source which makes it quite popular between the Individual and also for few big Organizations.**
- 5) **Browser Support: Dart Supported by all the browser because of its dart2js compiler that converts dart Program into JavaScript code which can be further processed by all the modern web browsers.**

Features of Dart Programming Language

- 5) Type Safe:** Dart is considered as type safe as it is the combination of static and runtime checks to ensure the variable value matches the variable type.
- 6) Flexible Compilation and Execution:** Dart Compilation is flexible as it supports both JIT (Just-in-Time) compilation as well as AOT (Ahead of Time) compilation. And dart2js adds extra value to it.
- 7) Asynchronous Programming:** Dart Supports Asynchronous Programming is where even when a Primary set of tasks are running. Program will respond to other set of tasks and at the end of the execution a final result will be returned.

a) JIT (Just-in-Time) Compilation

- Compiles code **while the app is running**
- Good for **development**
- Advantages:
 - Fast to start coding
 - Can see changes immediately (hot reload)
- Example: When using Flutter during development

b) AOT (Ahead-of-Time) Compilation

- Compiles code **before running**
- Good for **production apps**
- Advantages:
 - Faster execution
 - Smaller, optimized app
- Example: When building a Flutter app for release

So, combining JIT, AOT, and dart2js gives **flexibility**:

- JIT → Fast development
- AOT → High-performance mobile apps
- dart2js → Efficient web apps

Dart Programming Basics

1. Dart Comments

Dart Comments: In every programming language comments play an important role for a better understanding of the code in the future or by any other programmer. Comments are a set of statements that are not meant to be executed by the compiler. They provide proper documentation of the code.

Types of Dart Comments:

1. Dart Single line Comment.
2. Dart Multiline Comment.
3. Dart Documentation Comment.

1. Dart Single line Comment:

```
// This is a single line comment.
```

Example:

```
int main()  
{  
    double area = 3.14 * 4 * 4;  
    // It prints the area  
    // of a circle of radius = 4  
    print(area);  
  
    return 0;  
}
```

Output:

Output:

50.24

2. Dart Multi-Line Comment:

Example:

```
int main()
{
    var lst = [1, 2, 3];

    /*
    It prints
    the whole list
    at once
    */
    print(lst);

    return 0; }
```

```
/*

These are

multiple line

of comments

*/
```

Output:

```
[1, 2, 3]
```

3. Dart Documentation Comment:

Example:

```
bool checkEven(n){  
    /// Returns true  
    /// if n is even  
    if(n%2==0)  
  
        return true;  
    /// Returns false if n is odd  
    else  
  
        return false; }  
  
int main()  
{  
    int n = 43;  
    print(checkEven(n));  
    return 0;  
}
```

Output:

false

```
/// This is  
  
/// a documentation  
  
/// comment
```

In Dart, you use `///` or `/** ... */` for documentation comments.

```
/// This class represents a simple car
/// It has a color and can start
class Car {
  String color;

  /// Starts the car
  void start() {
    print("Car started");
  }
}
```

Example 2: Using `/** ... */`

```
/**  
 * This function adds two numbers  
 * and returns the result  
 */  
int add(int a, int b) {  
    return a + b;  
}
```

Explanation

1./// before class → Describes the class

2./// before variables → Explains what each variable stores

3./// before methods → Explains what the method does, including parameters and behavior

This is **professional style documentation**.

- Tools like **Dartdoc** can automatically generate a website showing all your classes and methods with these comments.

How Dartdoc Works

- Dartdoc reads all /// comments in your code.
- It creates a **HTML documentation website** automatically.
- Each class, variable, and method is shown with its description.


```
/// Represents a simple bank account
/// Stores account holder name and balance
class BankAccount {
    /// Name of the account holder
    String accountHolder;

    /// Current balance of the account
    double balance;

    /// Constructor to create a new bank account
    BankAccount(this.accountHolder, this.balance);

    /// Deposits [amount] into the account
    /// Increases the balance by the given amount
    void deposit(double amount) {
        balance += amount;
        print("\$$amount deposited. New balance:
\$$balance");
    }
}
```

```
/// Withdraws [amount] from the account
/// Decreases the balance by the given amount
void withdraw(double amount) {
    if (amount <= balance) {
        balance -= amount;
        print("\$$amount withdrawn. New balance: \$$balance");
    } else {
        print("Insufficient balance!");
    }
}

void main() {
    BankAccount myAccount = BankAccount("Walaa", 500.0);

    myAccount.deposit(200.0); // Adds 200
    myAccount.withdraw(100.0); // Withdraws 100
}
```

Class: BankAccount

Represents a simple bank account
Stores account holder name and balance

Constructor:

BankAccount(String accountHolder, double balance)

- Creates a new bank account with the given holder name and initial balance

Properties:

- String accountHolder
Name of the account holder

double balance

Current balance of the account

Methods:

- void deposit(double amount)
Deposits [amount] into the account
Increases the balance by the given amount
- void withdraw(double amount)
Withdraws [amount] from the account
Decreases the balance by the given amount
Prints "Insufficient balance!" if amount > balance

Why Use [parameterName]?

1. Makes the documentation **clearer**.
2. Helps tools like Dartdoc **create links automatically**.

2. Dart Variables

A variable name is the name assigned to the memory location where the user stores the data and that data can be fetched when required with the help of the variable by calling its variable name. There are various types of variable that are used to store the data. The type which will be used to store data depends upon the type of data to be stored.

To declare a variable:

```
type variable_name;
```

```
var name = 'Bob';
```

To declare multiple variables of the same type:

```
String name = 'Bob';
```

```
type variable1_name, variable2_name, variable3_name,  
....variableN_name;
```

Variable

Conditions to Write Variable Name

1. Variable names or identifiers can't be the **keyword**.
2. Variable names or identifiers can contain alphabets and numbers.
3. Variable names or identifiers can't contain spaces and special characters, except the **underscore** (`_`) and the **dollar** (`$`) sign.
4. Variable names or identifiers can't begin with a number.

***Note:** Dart supports **type-checking**, it means that it checks whether the data type and the data that variable holds are specific to that data or not.*

```
int age = 22;      // integer
double price = 19.99; // floating-point number
String name = "Walaa"; // text
bool isStudent = true; // true or false
```

- int → whole numbers
- double → decimal numbers
- String → text
- bool → true/false

Dart can **infer the type** using var:

```
var city = "Cairo"; // Dart knows it's String  
var score = 95;    // Dart knows it's int
```

Once inferred, the type **cannot change**.

```
var score = 95;  
score = 100; // valid  
score = "High"; // Error: type mismatch
```

Example

```
var name = 'Voyager I';  
var year = 1977;  
var antennaDiameter = 3.7;  
var flybyObjects = ['Jupiter', 'Saturn', 'Uranus', 'Neptune'];  
var image = {  
    tags : ['saturn'],  
    url : '//path/to/saturn.jpg',  
};
```

Type of the variable

1. Static Variable
2. Dynamic Variable
3. Final or const

1. Static Variable

```
void main() {  
    // Declaring and initializing a variable  
    int gfg1 = 10;  
    // Declaring another variable  
    double gfg2 = 0.2;    // must declare double a value or it  
                          // will throw error  
    bool gfg3 = false;   // must declare Boolean a value or it  
                          // will throw error  
    // Declaring multiple variable  
    String gfg4 = "0", gfg5 = "Geeks for Geeks";  
    // Printing values of all the variables  
    print(gfg1); // Print 10  
    print(gfg2); // Print 0.2  
    print(gfg3); // Print default string value  
    print(gfg4); // Print default bool value  
    print(gfg5); // Print Geeks for Geeks  
}
```

```
10  
0.2  
false  
0  
Geeks for Geeks
```



Thank
you!